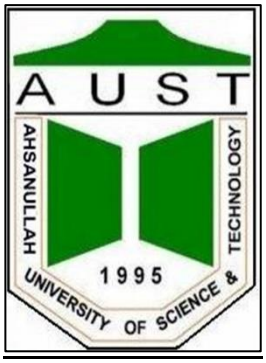
AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104
Course Title: Data Structures Lab

Assignment No: 01

Date of Submission: 29/06/2026

Submitted by,

Name: Hasibul Hasan
Student ID: 00724205101098
Lab Section: B2
Academic Semester: Fall 2025
Program: BSc in CSE

CSE2104 Offline-1

A bookstore maintains a sorted list of unique book IDs to quickly identify whether a book is available in stock. You are required to write a C++ program that uses **Binary Search** to determine if a specific book ID is present in the list. Additionally, the store often needs to find the first book ID that is not smaller than a given value (lower bound) and the first book ID that is greater than a given value (upper bound) in order to manage restocking and categorization. Given a sorted array of book IDs and a query book ID, your program should output whether the book exists in the list, the lower bound position and the upper bound position corresponding to the query.

Tasks

1. Read the number of book IDs and then read the sorted list of unique book IDs.
2. Read the query book ID to be searched.
3. Implement **Binary Search** to check if the book ID exists.
4. Implement logic to find the **lower bound** of the query.
5. Implement logic to find the **upper bound** of the query.
6. Display the search result along with the lower and upper bound indices.

You should write user defined functions for Binary Search, Lower Bound and Upper Bound. Input should be taken inside the main function. All user defined functions must be called from the main function.

Algorithm

1. Read the number of book IDs (N).
2. Read all the book IDs into a vector.
3. Sort the vector in ascending order using the sort() function.
4. Read the query book ID.
5. Use the Binary Search function to check whether the book ID exists.
6. Use the Lower Bound function to find the first index whose value is greater than or equal to the query.
7. Use the Upper Bound function to find the first index whose value is strictly greater than the query.
8. Display "Book Found" if the book exists; otherwise display "Book Not Found".
9. Display the Lower Bound index.
10. Display the Upper Bound index.

Source Code (C++)

```
#include <bits/stdc++.h>
using namespace std;
void binarySearch( vector<int> v, int key)
{
    int flag = 0;

    int high = v.size() - 1;
    int low = 0;
    int mid;

    sort(v.begin(), v.end());

    while(low <= high)
    {
        mid = (low + high) / 2;

        if(key == v[mid])
        {
            flag = 1;
            break;
        }
        else if(key > v[mid])
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    if(flag == 1)
    {
        cout << "Book Found";
    }
    else
    {
        cout << "Book Not Found";
    }
}

int lowerBound(vector<int> a, int key)
{
    int low = 0, high = a.size() - 1;
    int ans = a.size();

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (a[mid] >= key)
        {
            ans = mid;
```

```
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    return ans;
}

int upperBound(vector<int> a, int key)
{
    int low = 0, high = a.size() - 1;
    int ans = a.size();

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (a[mid] > key)
        {
            ans = mid;
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    return ans;
}

int main() {

    int n, x, key;
    vector<int> vec;

    cin >> n;

    for(int i = 0; i < n; i++)
    {
        cin >> x;
        vec.push_back(x);
    }

    cin >> key;

    binarySearch(vec, key);
    cout << endl;

    int lb = lowerBound(vec,key);
    cout << "Lower Bound Index: " << lb << endl;
```

```
    int ub = upperBound(vec,key);
    cout << "Upper Bound Index: " << ub << endl;

    return 0;
}
```

Sample Input	Sample Output
7 10 20 30 40 50 60 70 40	Book Found Lower Bound Index: 3 Upper Bound Index: 4